

PHP 기반 웹 취약점 탐지를 위한 Fuzzing 동향 및 향후 연구*

김민경*, 주경호**

*, ** 숭실대학교 (학부생, 교수)

Analysis of Fuzzing for Security in PHP Web Applications

Mingyeong Kim*, Kyungho Joo**

*, ** Soongsil University(Undergraduate student, Professor)

요약

본 논문은 Hypertext Processor(PHP) 기반 웹 애플리케이션의 취약점을 탐지하기 위한 Fuzzing 연구인 Phuzz, Atropos, FuzzCache의 특징과 한계를 비교 분석한다. 각 도구는 병렬 처리, 상태 스냅샷, 캐시 접근을 통해 탐지 효율을 개선하였으나, 특히 Phuzz, Atropos Fuzzer는 입력 생성에서 단순 변이 치중과 키-값 쌍 실시간 추론에 따른 오버헤드 발생의 구조적 한계를 보인다. 이에 Large Language Model(LLM) 능동 학습 기반의 입력 생성 방식을 도입한 새로운 연구 방향을 제안한다.

I. 서론

PHP는 웹 개발을 위한 서버 측 고수준 인터프리터 언어로, 전체 웹 애플리케이션 언어 74.3%의 높은 시장 점유율을 차지한다. 그러나 PHP 웹 애플리케이션은 전통적인 바 이너리 애플리케이션의 메모리 위반 중심 크 래시보다는 논리적 결함에 기반한 공격(SQL Injection(SQLi), Cross-site Scripting(XSS), Remote Code Execution 등)에 취약점을 드러낸다. 논리적 결함 취약점은 비정상적 동작, 정보 유출 등 심각한 보안 사고로 이어진다. 이러한 문제를 해결하기 위해 취약점 자동 탐지 기법인 Fuzzing이 주목받고 있다. 하지만 기존 Fuzzing 기법은 주로 바 이너리 프로그램을 대상으로 설계되어, PHP의 동적 특성과 고수준 언어 구조를 효과적으로 다루기 어렵다는 한계가 있다. 따라서 PHP 웹 애플리케이션의 특수성을 반영한 보안 중심의 Fuzzing 연구가 필요하다.

* 이 논문은 2025년도 정부(과학기술정보통신부)의 재원으로 정보통신기획평가원의 지원을 받아 수행된 연구임 (RS-2024-00442085, 자율주행 차량 서비스 보호를 위한 V2X 무선통신 인프라 보안 핵심기술 개발)

본 논문에서는 PHP 기반 웹 애플리케이션 보안을 위한 Fuzzing 연구 기술들의 특징과 한계를 분석하고, 효과적인 입력 생성을 위한 연구 발전 방향에 대해 제안한다.

II. 배경 지식

2.1 PHP 웹 애플리케이션 보안 특성

PHP는 URL, 폼, 쿠키, 세션 등 다양한 입력 채널을 처리하며, 세션 기반 상태 유지와 클라이언트-서버 간 복잡한 상호작용으로 인해 복합적인 취약점이 발생하기 쉽다. 또한 오픈소스 생태계 특성상 코드 품질에 편차가 크고, 입력 검증 실패와 같은 논리적 결함이 주요 공격 지점으로 작용한다. 이러한 특성은 Fuzzing 기법 적용 시 입력의 다양성, 상태 기반 추적, 다단계 흐름 분석 기능이 필수적임을 보여준다.

2.2 Fuzzing

Fuzzing은 무작위 또는 변형된 입력을 프로그램에 주입하여 비정상적인 동작이나 취약점을 자동으로 탐지하는 테스트 기법이다. Fuzzing은 내부 구조 정보 활용 여부에 따

[표 1] PHP 웹 취약점 탐지 Fuzzing 연구 비교

	Phuzz[1]	Atropos[2]	FuzzCache[3]
유형	커버리지 기반 그레이박스 Fuzzer		Fuzzer 성능 최적화 보조 도구
취약점 탐지	서버, 클라이언트 취약점 7개 탐지	서버 취약점 8개 탐지 (낮은 오탐)	대상 Fuzzer에 따라 상이
계측 대상	PHP 함수 런타임 환경	수정된 PHP 인터프리터	중복 데이터 감지 및 캐시
상태 관리	제한적 쿠키 공유	NYX 기반 전체 시스템 스냅샷 복원	공유 메모리 기반 캐시 일관성
특징	Docker 기반 다중 엔드포인트 병렬 처리	단일 인스턴스 심층 분석	데이터베이스, 네트워크 데이터 캐시 설계
강점	대규모 엔드포인트 경량 테스트	복잡한 상태 의존성 취약점 탐지	데이터 중복 요청 많은 시스템
한계	제한적 다단계 취약점 탐지	구현 복잡성	캐시 관리 필요

라 블랙박스, 그레이박스, 화이트박스로 분류되며, 본 논문은 코드 커버리지 피드백 정보를 활용하는 그레이박스 Fuzzer를 중심으로 분석한다.

III. PHP 웹 애플리케이션 Fuzzing

3.1 Phuzz[1]

Phuzz는 PHP 웹 애플리케이션의 서버, 클라이언트 측 복합 취약점을 탐지하기 위해 설계된 커버리지 기반 그레이박스 Fuzzer다.

본 도구는 HTTP 아카이브(HAR) 형식 초기 시드에 특수 공격 페이로드를 삽입하여 다양한 테스트케이스를 생성하고, Docker 기반의 모듈식 분산 아키텍처로 대규모 동시 테스트를 지원한다. 각 테스트케이스는 새로운 경로 발견 수에 따라 에너지를 할당하여, 변이 우선순위를 결정한다. 이러한 커버리지 피드백 방식과 다중 엔드포인트 병렬 처리를 통해 엔드포인트 간 상호작용을 신속하게 검증할 수 있다.

Phuzz의 핵심 기술은 API 호출을 중간에 가로채어 수정하는 후킹(Hooking) 기법이다. 함수 후킹으로 원본 코드 수정 없이 런타임 환경에서 위험함수 호출을 모니터링하거나 인증우회로 심층적인 프로그램 동작 분석이 가능하다. 실제 115개 인기 WordPress 플러그인의 1000개 이상 API 엔드포인트 대규모 테스트에서 20건 이상의 보안 이슈와 2개의 새로운 CVE 취약점 발견 성과를 보였다.

그러나 Phuzz는 후킹 기반의 계측에 의존하기 때문에 오탐의 가능성이 존재하며, 후킹된 함수를 호출할 시 성능 오버헤드가 발생하여 전체 요청 처리 시간 저하가 수반된다. 또한 세션 정보를 지속적으로 유지하기 어려워 다단계의 연쇄적 상호작용을 요구하는 취약점 탐지에는 한계가 있다.

3.2 Atropos[2]

Atropos는 오탐률을 최소화하며 PHP 기반 웹 애플리케이션의 서버 취약점을 탐지하는 동적 Fuzzer다.

본 도구는 추론과 최적화 과정을 거쳐 키-값 쌍이 구조화된 입력을 생성한다. 키-값 쌍은 특정 페이지 접근을 돕는 중요한 토큰 정보로, 피드백 결과에 따라 유효한 입력을 생성함으로써 심층 로직 테스트가 가능하다.

Atropos의 핵심 기술로 NYX 기반의 스냅샷 상태 관리가 있다. 이를 통해 데이터베이스 구조를 포함한 시스템 정보를 빠르게 초기화함으로써 단일 인스턴스를 심층적으로 분석할 수 있고 동일 입력에 대해 일관된 취약점을 도출할 수 있다. 실제로 미인증 사용자에서 발생 가능한 새로운 취약점 7개를 발견하는 성과를 나타냈다.

그러나 Atropos는 시스템 설계 및 구현 복잡도가 높고 메모리 및 자원 소모가 커 경량의 정적인 웹사이트 취약점 탐지 목적으로는 부적합하다는 단점이 있다.

3.3 FuzzCache[3]

FuzzCache는 기존에 존재하는 PHP 웹 애플리케이션 대상 Fuzzer의 성능 최적화를 돕는 보조 도구다. 캐시 정책을 설계해 데이터베이스와 네트워크의 반복 요청으로 인한 성능 오버헤드를 해결한다.

본 도구는 두 가지 주요 캐싱 메커니즘을 통해 테스트 효율성을 보인다. 첫째, 데이터베이스 캐시는 쿼리 문법으로 읽기·쓰기 연산을 구분한다. 읽기 쿼리 요청 시 데이터베이스 연결을 지연한 후 캐시 적중 여부에 따라 관련 데이터를 가져오거나 새롭게 저장하는 반면, 쓰기 쿼리 요청 시에는 테이블 단위로 캐시의 연관 데이터 전체를 무효화한다. 둘째, 네트워크 캐시는 Time-to-Live(TTL)을 계산해 캐시된 데이터에 유효시간을 부여함으로써 정보의 최신성을 보장한다.

FuzzCache는 독립적으로 사용되는 도구가 아니며, 외부 Fuzzer 도구(Black-Window, WebFuzz 등)와 함께 사용되어야 한다. 실험 결과, 기존 시스템 대비 약 3.3배에서 최대 3.8배까지 처리량이 증가하는 성능 향상 효과를 보였으며, 데이터베이스 및 네트워크 요청이 빈번한 웹 애플리케이션 환경에서 높은 효율성을 나타냈다.

그러나 FuzzCache는 테이블 단위의 캐싱 방식으로 인해 불필요한 데이터까지 메모리에 로드되어 비효율적이며 동적 취약점 탐지 정확도가 저하될 수 있다는 한계가 있다.

IV. 결론 및 향후 연구

본 논문에서는 PHP 웹 애플리케이션의 취약점을 동적으로 탐지하는 Fuzzing 기법들을 분석하고 표 1에 정리하였다. Phuzz는 다중 엔드포인트 병렬 처리, Atropos는 상태 의존적 취약점 탐지에 강점을 가지며, FuzzCache는 성능 최적화를 통해 처리량을 향상시킨다.

그러나 현재 기술들은 입력 생성 기법 측면에서 한계를 지닌다. Phuzz는 시드 기반의 단순한 입력 변이에 의존하고 있으며, Fuzz

Cache는 키-값 쌍 기반의 입력 구조를 동적으로 추론하지만, 그 과정에서 높은 자원 소모가 수반된다. 이러한 한계는 복잡한 상태 기반 취약점이나 다단계 상호작용 탐지에 있어 탐색 효율성 저하로 이어질 수 있다.

이에 따라, PHP 웹 취약점 탐지 Fuzzing 연구에 새로운 입력 생성 기법을 결합하는 것이 중요하다. 대표적으로 LLM의 고도화된 자연어 처리 능력과 능동 학습 전략을 기반으로, 웹 애플리케이션의 입력 구조와 실행 흐름을 이해하여 정교한 상태 기반 입력 생성을 구현할 수 있다. 이는 복잡한 입력 패턴 학습과 다단계 상호작용 예측에 강점을 가지며, 기존 Fuzzer의 탐지 한계를 보완한다. 또한 코드 분석 기반의 목적성 있는 입력 생성은 무작위 생성 방식 대비 효율적인 테스트와 연산 비용 절감이 가능하다. 이러한 접근은 Fuzzing 탐지 효율성과 현실 적용 가능성을 크게 향상시킬 것으로 기대된다.

[참고문헌]

- [1] Sebastian Neef, Lorenz Kleissner and Jean-Pierre Seifert. “What All the Phuzz Is About: A Coverage-guided Fuzzer for Finding Vulnerabilities in PHP Web Applications” Proceedings of the 19th ACM Asia Conference on Computer and Communications Security. July 2024.
- [2] Emre Güler, et al. “Atropos: Effective Fuzzing of Web Applications for Server-Side Vulnerabilities”. Proceedings of the 33rd USENIX Security Symposium. August 2024.
- [3] Penghui Li and Mingxue Zhang. “FuzzCache: Optimizing Web Application Fuzzing Through Software-Based Data Cache” Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. December 2024.